Learned Index or B+-Tree? An End-to-End Comparison in an Embedded SQL Engine

Aadit Sood

***Abstract*—Learned indexes answer point lookups faster than B-trees when measured on their own, but a database also has to parse and plan each statement, log each write, and turn index entries back into rows. We built a small embedded SQL engine whose storage engine is a learned index (piecewise-linear segments over a gapped array, with packed-memory-array rebalancing), and made the index swappable at compile time for a well-known B+-tree library with nothing else changed, write-ahead log and crash recovery included, and measured both builds on two machines. On its own, the learned index is 1.5–1.9x faster than the B+-tree at 1M keys, 1.3–1.5x at 10M, and 1.1–1.5x on four real datasets. Inside the engine the advantage shrinks to 1.06–1.20x for a query that was parsed ahead of time and to about even (0.99–1.06x) for one that is parsed each time. Range scans and random-order inserts are even; appends are 1.5–3.4x slower on the learned index, and inserts that do not follow the key distribution are up to 3.3x slower. Durable writes are dominated by fsync whichever index is used, even with batching; opening the database from a snapshot is 1.4–1.7x faster on the learned index, and memory per row is about the same. On one machine, the lookup speedup at 10M keys ranged from 1.02x to 1.47x between runs of the same program, which interleaved measurement within a process cannot detect. We also describe two performance problems that stayed hidden while the index was compared only with weak baselines, and the differential testing against SQLite that both builds pass.**

I. Introduction

A learned index replaces the traversal of a B-tree with a model that predicts where a key sits in a sorted array, followed by a short local search to correct the prediction [1]. Measured on their own, learned indexes answer lookups faster than B-trees and need far less memory for their routing structure [2]. A database does much more than look up keys, though. It parses and plans each statement, logs each write before applying it, copies rows in and out of its storage, and has to recover after a crash. How much of the index's advantage is left once all of that is added is a different question from how fast the index is.

Several systems have tried to answer it for their own setting. Bourbon put learned models into the files of an LSM-tree key-value store [3], Google measured learned indexes in a disk-based database [4], and PostLearn integrated the ALEX+ learned index into PostgreSQL as an index access method [5]. PostLearn is the closest to this paper. It reports speedups of up to 1.5x for point lookups inside PostgreSQL, smaller ones once the client is included, and it supports reads only; updates, write-ahead logging and crash recovery are left as future work.

This paper studies those missing parts in a much smaller setting: an embedded SQL engine in the style of SQLite, built from scratch, whose storage engine is a learned index. The engine has a recursive-descent SQL parser, an executor that routes each query to a point lookup, range scan, secondary-index lookup or full scan, a catalog stored as a table, a checksummed write-ahead log, and snapshot checkpoints. It is built so that the learned index can be replaced, at compile time, by a well-known B+-tree library (tlx [6]) with nothing else changing. Both builds pass the same test suites and the same differential test against SQLite [7]. That makes it possible to measure what the learned index is worth at each layer of the engine and for each kind of operation: point lookups with and without parsing, range scans, inserts in random and increasing order, inserts that follow or depart from the existing key distribution, durable writes with and without batched fsync, recovery time, and memory per row.

Most of that advantage does not survive inside the engine. The learned index is 1.5–1.9x faster than tlx at the storage layer on 1M keys, but at that size a point query through SQL is only 6–9% faster when it was parsed ahead of time, and about even when it is parsed each time, mostly because the lookup is a small part of what the query costs. Range scans and random inserts are even. The learned index's known weaknesses, on the other hand, carry through: appends are 1.5–3.4x slower, and inserts that land where the data is sparse are up to 3.3x slower. Durable writes are dominated by fsync on either index. The learned index's largest gain inside the engine is on lookups for ids that are not in the table (1.7–2.5x), and it also gains in startup time and in read-heavy workloads after inserts, where the B+-tree's split leaves slow it down.

None of the index's techniques are new, and the paper does not claim them. The segmentation is the greedy method of FITing-tree [8], the gapped array comes from ALEX [9], and the rebalancing scheme is that of a packed-memory array [10], [11]. The contributions are:

- an embedded SQL engine whose index can be swapped between a learned index and a B+-tree with everything else held fixed, with the write-ahead log, checkpointing and recovery included (Section III);
- a measurement of the learned index's value inside that engine across reads, writes, durability, recovery and memory, on synthetic data and on four real datasets from the SOSD benchmark (Section V);
- an account of two performance problems that stayed

invisible while the index was compared only against weak baselines, and of the testing that caught correctness bugs (Section VI).

The code, the benchmark scripts and the raw output of every run reported here are public.

II. Background and Related Work

a) Learned indexes: Kraska et al. proposed replacing index structures with models that learn the cumulative distribution of the keys [1]. FITing-tree splits the keys into linear segments with a guaranteed maximum error, fitting them greedily in one pass [8]. The PGM-index computes the optimal segmentation for a given error bound with O'Rourke's algorithm [12] and indexes the segments recursively [13]. RadixSpline builds a spline and a radix table in a single pass [14]. ALEX made learned indexes updatable by storing keys in gapped arrays, with room left for inserts, and adapting its node layout to the data [9]. SOSD compares these and other structures on real datasets [2].

b) Evaluations of updatable learned indexes: Wongkham et al. evaluated updatable learned indexes under many workloads and found their advantages depend heavily on the workload and the data [15]. Lan et al. found that once data lives on disk, a B+-tree is competitive with them in most settings [16].

c) Learned indexes in systems: Bourbon reported 1.23x to 1.78x faster lookups in an LSM-tree store [3]. Abu-Libdeh et al. describe learned indexes in Bigtable [4]. PostLearn, as described above, measured ALEX+ inside PostgreSQL for reads [5].

d) Robustness: Learned indexes can be attacked through the data they are built on. Kornaropoulos et al. poisoned the training data of recursive model indexes [17]; Bachfischer et al. measured slowdowns of up to 20% from poisoning in ALEX and the PGM-index [18]; and Yang et al. built insert sequences that slow ALEX down by up to 1,641x or exhaust its memory [19].

e) Testing and measuring: Differential testing of SQL engines goes back to RAGS [20]; SQLancer applies related ideas at scale [21]. Mytkowicz et al. showed how innocuous-looking setup changes bias performance measurements [22], which is the reason for the interleaved measurements in Section IV.

III. The Engine

A. Overview

The engine is single-process and keeps all rows in memory, like an in-memory SQLite database with a log. A statement is tokenized and parsed by a recursive-descent parser; the executor turns an equality condition on the primary key into a point lookup, `BETWEEN` on the primary key into a range scan, an equality condition on an indexed column into a secondary-index lookup, and anything else into a full scan with filtering. The storage layer maps 64-bit keys to byte strings. Each row's key combines a 16-bit table id with its 48-bit primary key, so a table's rows are contiguous and ordered; secondary-index entries combine a 20-bit value with a 28-bit primary key and live in the same store. The catalog is a table (id 0) and rebuilds itself on startup by scanning it.

Every write is appended to the write-ahead log, with a CRC-32 per record, before it is applied. Opening the log truncates it to its last intact record. A checkpoint writes the whole store to a snapshot (to a temporary file, fsync'd and renamed) and then empties the log. On startup the engine loads the snapshot, folds the log into it (for each key, the last record logged wins), and builds the index once from the result.

B. The learned index

The index stores keys in a gapped array that starts 70% full. A model of linear segments predicts each key's slot. Segments are fit in one pass: each segment's line is pinned to its first key, and the segment grows while some slope keeps every key within $\epsilon = 64$ slots of its prediction. This is the greedy method of FITing-tree [8]; an optimal segmentation [12], [13] is also implemented but not used, because the index refits regions often and the optimal fit is four to seven times slower to compute. Segments are capped at 4,096 keys, because on perfectly regular keys a single line would fit the whole array and refreshing that one segment would mean rebuilding everything.

A lookup finds the segment with a branchless binary search over the segments' first keys, computes the prediction, clamps it to the segment's stretch of the array, and walks from there toward the first key that is not smaller than the target, skipping empty slots. The walk takes at most ϵ steps in each direction before switching to an exponential search, so the worst case is logarithmic. Correctness never depends on the model: because the array is sorted, the walk finds the right slot from any starting point, and the model only decides how long the walk is.

C. Inserts

An insert shifts keys toward the nearest empty slot, which moves each shifted key one slot away from where the model predicts it. Each segment counts the inserts that changed it, meaning the segment that received the key and any segment whose keys were shifted, and is refit once its count reaches ϵ. Since each counted insert moves a key by at most one slot, every key the model was fit on stays within 2ϵ of its prediction. When a segment's region is too full to refresh (90%), the refresh widens to aligned windows of two, four, eight segments, with density limits that fall from 0.9 for one segment to 0.8 for the whole array, as in a packed-memory array [10], [11]. The whole array is rebuilt at a larger size only when it is more than 80% full. The last segment can instead grow the array at its end, by a quarter of its size at a time, which is what keys arriving past the end use.

D. Batched fsync

By default every log record is fsync'd before the write returns. With a setting $k > 1$, only every k-th record is fsync'd, and the log is synced on close and before a checkpoint

empties it. Every record still reaches the operating system before the write returns, so a crash of the process loses nothing; up to $k - 1$ acknowledged writes can be lost if the machine loses power. SQLite makes a similar trade with `synchronous=NORMAL`. This is not group commit in the usual sense, in which concurrent clients share an fsync and each still waits for it: the engine has one client and no multi-statement transactions.

E. The B+-tree build

The storage layer refers to its index through a single type alias. A compile-time switch replaces the gapped array with a thin wrapper around tlx's `btree_map` [6]. tlx stores each row next to its key in the tree's leaves, while the gapped array keeps rows in a separate array. This build uses tlx's default node settings, which here means 8 rows per leaf and 16 children per inner node; we did not tune them for the engine. tlx's B+-tree is the successor of the stx B+-tree used by SOSD. The parser, executor, catalog, log and snapshot code are identical in both builds; the only other difference is the text of `EXPLAIN`. Both builds pass the same unit and end-to-end tests and agree with SQLite on the same randomized differential test.

IV. Methodology

a) Machines: We used two machines with the same operating system (Ubuntu 26.04) and compiler (g++ 15.2, `-O2`). The *laptop* has an Intel Core i7-13620H (6 performance and 4 efficiency cores, 24 MB of L3 cache) and runs Linux 6.18 under WSL2 with 7.6 GB of memory available to it, with nothing else running. *AWS* is a `c7a.xlarge` virtual machine with an AMD EPYC 9R14 (4 vCPUs, 7.5 GB of memory) running Linux 7.0. Neither is bare metal, and on AWS other tenants can share the processor's cache. Threads were not pinned to cores. For CPU-bound tests the log is written to a tmpfs, where fsync costs nothing; the durability tests write to an ext4 file system, on a local SSD on the laptop and on network storage (EBS gp3) on AWS, so their absolute times are not comparable between the machines.

b) Data: Synthetic keys are uniform (1M and 10M keys, drawn from $[0, 10n)$) and skewed (lognormal with $\sigma = 2$, scaled to 10^{15}). Real keys come from four SOSD datasets [2]: books, fb, wiki and osm, each downsampled by keeping every k-th key to 10M keys, the way SOSD itself shrinks its larger datasets, and checked against the checksums SOSD publishes. wiki contains repeated timestamps, of which 2,417 were dropped to make the keys unique; osm's keys were shifted right one bit so they fit in a signed 64-bit integer. For SQL-level tests, keys too large for a 48-bit primary key are shifted right until they fit, which keeps their order and the shape of the distribution.

c) Procedure: Every structure's answers are checked before it is timed. Within one program, each pass times every structure once, in turn, and results are medians over seven passes; an earlier version that timed structures one after another produced differences of more than 10% for the same structure depending on when it ran. The two builds of the engine are separate programs, so each engine experiment on synthetic data runs three times per build, alternating, and reports medians; the real datasets run once per build. On the laptop the same measurement moves by 10–20% between runs, so differences smaller than that are not meaningful; AWS was much steadier. One script reruns every experiment and records the machine, compiler and commit next to the raw output.

d) Variation between processes: Interleaving only controls for differences within one process. To see how much remains between processes, we ran the 10M uniform lookup benchmark 14 times on the laptop in one sitting, as separate processes. The speedup over tlx ranged from 1.11x to 1.47x, in two clusters (1.11–1.20x and 1.34–1.47x), mostly because the gapped array took either about 290 or about 350 ns. Both clusters appeared with transparent huge pages disabled and with the default setting, so huge pages do not explain it, and the cause is unknown. A run of the full script five days earlier on the same laptop, with the same index and lookup code, measured 1.02–1.08x at 10M keys, so the observed range is 1.02–1.47x. We therefore read the laptop's 10M lookup speedups as a range. We did not repeat this experiment on AWS, though its separate engine runs agree closely.

V. Results

All numbers in this section come from two runs of the benchmark script at commit `ab67357`, one per machine. Speedups are the B+-tree's time divided by the learned index's, so values above 1 favor the learned index. At the storage level (Table I, Table III), "tlx" is tlx's B+-tree with 512-byte nodes, its fastest setting for lookups on the laptop among node sizes from 128 bytes to 2 KB; we did not repeat that tuning on AWS. The whole-engine comparison (Table II) uses tlx's default node settings instead (Section III-E).

A. The index on its own

Table I times point lookups on the gapped array the engine uses, freshly built on 90% of each dataset, against tlx bulk-loaded on the same keys and against binary search over a sorted array. The learned index is 1.5–1.9x faster than tlx at 1M keys and 1.3–1.5x at 10M, where neither fits in the processor's cache, subject to the variation described in Section IV. The two machines agree on the real datasets' order: books gains the most (1.39x and 1.54x), then wiki, and fb and osm the least (1.10–1.13x). fb and osm are the two that need many more segments (their models are five to twelve times larger per key than the other two real datasets'), which matches their reputation as hard datasets for learned indexes [2]. On AWS, binary search over a plain array is faster than tlx, so tlx may not be at its best there; the learned index is still 1.34–1.69x faster than binary search on the synthetic data, but only 1.02–1.03x on fb and osm. A sorted array cannot take inserts, though, and the gapped array and tlx can.

After the remaining 10% of the keys are inserted, the learned index is 1.2–2.0x faster than tlx on every dataset: tlx's leaves start out full and split, while the gapped array had room set aside. The learned model takes 0.006–0.07 bytes per key,

TABLE I
POINT LOOKUPS ON THE INDEX ALONE (NS, MEDIANS OF SEVEN INTERLEAVED PASSES) AND THE LEARNED MODEL'S SIZE.

	Laptop				AWS				
Data	Learned	tlx	Binary	Speedup	Learned	tlx	Binary	Speedup	Model (B/key)
uniform 1M	115.3	178.3	167.7	1.55	78.4	145.6	132.3	1.86	0.0065
skewed 1M	120.8	185.7	167.6	1.54	84.7	147.3	135.6	1.74	0.0074
uniform 10M	293.0	390.2	406.1	1.33	234.1	343.6	312.6	1.47	0.0064
skewed 10M	288.5	390.4	403.9	1.35	233.0	338.2	311.7	1.45	0.0066
books 10M	286.2	398.8	433.0	1.39	220.0	337.7	314.6	1.54	0.0060
fb 10M	395.1	433.9	469.6	1.10	298.4	335.7	304.5	1.13	0.0695
osm 10M	370.8	418.7	434.5	1.13	303.0	336.0	313.4	1.11	0.0723
wiki 10M	335.9	412.4	468.2	1.23	266.0	337.8	310.0	1.27	0.0147

against 0.25 for tlx's inner nodes (0.53 with its default 256-byte nodes), but the gapped array as a whole takes 11.4–11.5 bytes per key, against 8.6 for the packed tlx tree, because a third of its slots are empty. Against the published PGM-index on 1M keys [13], the pinned segmentation over a dense array, without the gapped array's empty slots or its cap on segment length, took 89.4 and 92.8 ns on uniform and skewed data on the laptop, and the PGM-index 92.9 and 112.1 ns; on AWS, 72.1 and 75.4 ns vs. 70.9 and 87.7 ns. We read that as even on uniform data and in the pinned segmentation's favor on skewed data.

B. Reads inside the engine

Table II compares the two builds of the whole engine on a table of `(id INT, val INT)` rows. The store's own lookup, timed in a separate loop and including copying the row out, is 1.1–1.6x faster on the learned index. Once the executor has planned the query and decoded the row, the speedup is 1.06–1.20x, and once the statement has to be parsed it is about even (0.99–1.06x). Much of that is arithmetic: on AWS at 1M keys a parsed query takes about 1,300 ns, of which the store's lookup is about 280 ns. There the saving simply carries through, since the store's lookups differ by 50 and 35 ns on uniform and skewed data and the pre-parsed queries by 43 and 35 ns. It does not carry through everywhere, though: at 10M uniform keys on AWS it shrinks from 173 to 112 ns, and on fb from 130 to 18 ns. Parsing shrinks it further and sometimes reverses it, although a fixed extra cost should leave it unchanged, and we do not know why. Lookups for ids that are not in the table are 1.7–2.5x faster on the learned index on every dataset and on both machines; this too is unexplained. Range scans are even at every length (0.92–1.13x at 10 rows, 0.97–1.06x at 100 and 1,000), since decoding the rows dominates.

C. Writes

Random-order inserts through SQL are even (0.96–1.03). At the storage level the gapped array takes 375 and 408 ns per insert on uniform and skewed 1M keys on the laptop, vs. 277 and 322 ns for tlx, and 336 and 347 ns on AWS vs. 271 and 263 ns (each a single timed pass, so noisier than the lookups); that difference disappears in the 2.4–4.0 μs an insert costs through SQL. Appends past the end of the key range, the pattern of an auto-increment primary key, are 1.5–3.4x slower on the learned index. At the storage level the gap is larger, 11 to 15 times (415–553 ns vs. 36–37 ns for tlx), because a B+-tree appends by touching only its rightmost leaf, while the gapped array keeps shifting keys at its end and refreshing its last segment.

The mixed workloads separate two kinds of insert. When new ids are drawn between existing ones, so that they follow the data's distribution, a read-heavy mix (95% reads) is 1.16–1.45x faster on the learned index, which is consistent with tlx's lookups slowing down as its leaves split. When new ids are drawn evenly from the whole key range, in the 50/50 mix, the learned index is 1.6–1.9x slower on skewed data. Table III shows why at the storage level. Skewed data leaves a few segments that cover most of the key range but hold few keys; evenly spread inserts pour into their small regions, which fill at once and keep borrowing room from their neighbors, so each insert rewrites far more of the array.

These mixes have a flaw: they run one after another on the same table, so the evenly spread mix starts with about 110,000 more rows than the first mix did, about a tenth of the table at 1M keys and 1% at 10M. That pushes the gapped array closer to its density limits. It is a likely reason the two 50/50 mixes differ on uniform 1M keys (1.02 vs. 0.78 and 0.86), where their inserts should behave alike, so the spread-evenly figures in Table II should be read with that in mind, especially at 1M keys. Table III builds fresh structures for each order and does not have this problem, and it shows the same effect on skewed data.

On uniform keys the two kinds of insert draw from the same distribution, and the times match on both machines. Everywhere else, spreading the inserts makes the gapped array 1.6–3.4x slower, raises the slots it rewrites per insert by up to 43 times, and forces refreshes that borrow from neighboring segments (between 24 and 1,814 of them, against none when the inserts follow the data). tlx stays about the same on books (1.02–1.12x faster) and gets 1.2–3.1x faster on the others, presumably because the new keys land in a few leaves that stay in cache.

On 900K uniform keys, concentrating 100K inserts into a narrow key range (1.2–5% of it) made them 57–80% slower

TABLE II

THE WHOLE ENGINE ON EACH INDEX, WITH TLX AT ITS DEFAULT NODE SETTINGS: THE B+-TREE'S TIME OVER THE LEARNED INDEX'S, FROM MEDIANS OF THREE ALTERNATING RUNS OF EACH BUILD. IN EVERY COLUMN, MEMORY PER ROW AFTER OPENING FROM A SNAPSHOT IS 89.2 BYTES ON THE LEARNED INDEX AND 94.3 ON TLX.

	Laptop			AWS		
Operation	uniform 1M	skewed 1M	uniform 10M	uniform 1M	skewed 1M	uniform 10M
store lookup (row copied out)	1.30	1.60	1.53	1.18	1.12	1.43
point query, parsed ahead of time	1.08	1.09	1.20	1.08	1.06	1.15
point query, parsed each time	1.02	1.06	1.03	0.99	1.02	1.02
point query, missing id	2.31	2.26	2.22	2.46	2.32	2.09
range scan, 100 rows	1.01	0.98	0.99	1.02	0.99	1.02
insert, random order	0.96	0.97	1.00	1.00	0.99	1.03
insert, appended	0.54	0.53	0.30	0.67	0.67	0.43
95% reads, 5% inserts following the data	1.17	1.17	1.45	1.17	1.16	1.29
50% reads, 50% inserts following the data	1.02	0.99	1.11	1.02	1.00	1.09
50% reads, 50% inserts spread evenly	0.78	0.52	1.25	0.86	0.64	1.11
durable insert, fsync every write	1.43	0.94	1.10	1.00	1.00	1.00
durable insert, fsync every 64 writes	1.43	1.25	1.22	0.99	0.99	0.99
open from a snapshot	1.56	1.65	1.42	1.74	1.74	1.45

TABLE III

INSERTING 10% MORE KEYS AT THE STORAGE LEVEL, EITHER DRAWN FROM THE DATA OR DRAWN EVENLY FROM THE WHOLE KEY RANGE, IN RANDOM ORDER (NS PER INSERT, LEARNED / TLX). THE LAST COLUMN GIVES THE SLOTS THE GAPPED ARRAY REWRITES PER INSERT, FROM THE DATA → SPREAD EVENLY, WHICH DO NOT DEPEND ON THE MACHINE. EACH IS A SINGLE TIMED PASS.

	Laptop		AWS		
Data	From the data	Spread evenly	From the data	Spread evenly	Slots rewritten
uniform 1M	375 / 277	375 / 273	336 / 271	333 / 265	49 → 48
skewed 1M	408 / 322	1,066 / 145	347 / 263	898 / 116	49 → 161
books 10M	545 / 518	934 / 461	487 / 459	820 / 450	47 → 77
fb 10M	480 / 519	1,318 / 201	360 / 463	1,238 / 151	3 → 129
osm 10M	498 / 536	1,043 / 283	376 / 458	930 / 248	5 → 85
wiki 10M	569 / 513	898 / 402	504 / 456	830 / 384	35 → 71

than spreading them out (536–643 vs. 330–374 ns; medians of nine interleaved rounds), more so as the range narrows, with no full rebuilds. This is a far weaker attack than published ones on learned indexes [19]; we did not attempt a targeted one.

D. Durability

With an fsync on every write, the index makes no measurable difference. On AWS, where nearly every run's durable insert took 2.7 ms, the medians on the two indexes match to within 1% on all seven datasets. The laptop's SSD is much noisier: medians of 2.0–2.9 ms, with single runs from 1.6 to 4.0 ms, so its speedups in both durable-insert rows of Table II reflect the disk rather than the index. Batching helps as expected: on 1M uniform keys, fsyncing every 8, 64 and 512 writes brings the learned index's insert to 343, 47 and 9.1 μs on AWS (343, 47 and 8.8 μs for tlx), approaching the 3.6 μs an insert costs there with no fsync at all.

E. Startup and memory

Opening the engine from a snapshot is 1.4–1.7x faster on the learned index (129 vs. 201 ms at 1M rows on the laptop, 1.6 vs. 2.3 s at 10M), and from the log, which the engine folds into the snapshot's rows before a single build, 1.35–1.57x faster. Laying out a sorted array is cheaper than building a tree; the B+-tree wrapper also makes a second pass to move the rows into the leaves, because tlx's bulk load copies its input. Memory per row is about the same once both are built from a snapshot (89 vs. 94 bytes; most of a row is the row itself), and 23–30% lower on the learned index after a load by random-order inserts, which leaves tlx's leaves partly empty.

F. Real datasets

On the four SOSD datasets (one run of each build at 10M keys, so noisier than the medians above), a pre-parsed point query ranged from even to 1.20x faster on the learned index, a parsed one from 10% slower to 7% faster, appends were 2.3–3.4x slower (speedup 0.30–0.43), the read-heavy mix following the data was 1.19–1.59x faster, and inserts spread evenly over the key range ranged from 1.22x faster (books) to 3.3x slower (fb).

VI. LESSONS FROM BUILDING IT

a) A weak baseline hid quadratic inserts: For most of the project's life, the insert benchmark compared the gapped array with inserting into a plain sorted vector, which shifts every later key, and reported a twelvefold advantage. Compared later

against tlx, inserts were 45 to 1,200 times slower than the B+-tree, and the gap grew with the array. The cause was a safety net: a full rebuild every 512 inserts regardless of the array's size, which made the average insert linear in the number of keys. It existed because the drift accounting ignored shifts that crossed into a neighboring segment. Counting those, adding windowed rebalancing and growth at the end, and capping segment length removed every full rebuild from the benchmarks.

b) A third of every lookup was a mispredicted branch: Finding a key's segment used `std::upper_bound`, whose branch goes either way with equal probability for random keys. Rewriting the binary search to select each half with a conditional move cut it from about 48 to about 9 ns, a third of the lookup time at 1M keys, and changed the comparison with the PGM-index from a clear loss on uniform data to a tie.

c) An invariant's test must fail without the mechanism: A test of the 2ϵ drift bound passed with the neighbor counting removed. Each shift goes toward the nearest gap, so keys rarely move far into a neighboring segment unless it has a long run of empty slots at its edge. Only after the test deleted keys next to a segment boundary, which creates such a run, did it fail without the counting (worst error 2.47ϵ) and pass with it.

d) Differential testing found what unit tests did not: Running random statements against both this engine and SQLite, with automatic shrinking of failing cases, found a `LIMIT` applied before `COUNT(*)`, secondary-index entries left behind by inserts and updates that replaced rows, and out-of-range values silently truncated into other keys. The same test now runs on both builds of the engine, and with a tiny error bound ($\epsilon = 4$) that makes the rebalancing code run constantly.

e) The best number came from a broken dataset: An early skewed dataset collapsed into a run of consecutive integers, which a single line fits exactly, and produced the project's fastest lookups. Looking at the gaps between keys would have caught it immediately.

VII. Limitations

All results come from two machines, neither of them bare metal: a laptop under WSL2 and an AWS virtual machine that may share its cache with other tenants, on network storage, with threads not pinned to cores. We did not establish what causes the variation between processes on the laptop, or test for it on AWS. tlx's node size was tuned only on the laptop and only at the storage level; the engine comparison uses its default settings. The engine's mixed workloads share one table, so later ones start with more rows. The engine keeps everything in memory and reads its whole snapshot on startup, unlike disk-based engines that load pages on demand. It has one client, no multi-statement transactions and no joins. Only one B+-tree (tlx) and one learned index design are compared; an updatable learned index such as ALEX inside the same engine would be a natural next comparison. The real datasets are downsampled to 10M keys, and SQL-level tests shift large keys to fit 48 bits. Because the two builds are separate programs, their comparison relies on alternating runs rather than interleaved passes.

VIII. Conclusion

Swapping only the index under one engine shows how much of a learned index's advantage reaches the user of a small embedded database: at 1M keys, a 1.5–1.9x faster lookup at the storage layer becomes a 6–9% faster query when it was parsed ahead of time, and about even when it was not, while the learned index's weaknesses on appends and on inserts that depart from the data's distribution carry through. The two machines agree on this, even where their absolute times differ. For an engine like this one, a learned index is not a clear win over a B+-tree library, even one left at its default node settings; tuning the engine's tree might narrow the remaining gap, which we did not test. It trades slightly faster lookups, faster startup and better behavior after random inserts for much slower appends and a sensitivity to where inserts land. The obvious next steps are an updatable learned index such as ALEX in the same engine, a disk-resident version, and measurements on bare metal.

Artifact

The code, the scripts that produce every number in this paper, and the raw output of the runs reported here are at https://github.com/asood8/Learned-Index; the runs reported here were made at commit `ab67357`, and their output is in `results/runs/2026-10-31_0201` (laptop) and `results/runs/2026-10-31_0514` (AWS). The variation experiment is in `results/experiments`, and the earlier laptop run in `results/runs/2026-10-26_1522`.

References

[1] T. Kraska, A. Beutel, E. H. Chi, J. Dean, and N. Polyzotis, "The case for learned index structures," in *Proceedings of the 2018 International Conference on Management of Data (SIGMOD)*, 2018.

[2] R. Marcus, A. Kipf, A. van Renen, M. Stoian, S. Misra, A. Kemper, T. Neumann, and T. Kraska, "Benchmarking learned indexes," *Proceedings of the VLDB Endowment*, vol. 14, no. 1, 2020.

[3] Y. Dai, Y. Xu, A. Ganesan, R. Alagappan, B. Kroth, A. C. Arpaci-Dusseau, and R. H. Arpaci-Dusseau, "From WiscKey to Bourbon: A learned index for log-structured merge trees," in *14th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2020.

[4] H. Abu-Libdeh, D. Altınbüken, A. Beutel, E. H. Chi, L. Doshi, T. Kraska, X. Li, A. Ly, and C. Olston, "Learned indexes for a Google-scale disk-based database," NeurIPS Workshop on ML for Systems, arXiv:2012.12501, 2020.

[5] A. Fuad, S. Vashisth, O. Balmau, and B. Kemme, "PostLearn: Towards a learned index for PostgreSQL," in *Workshop on Challenges and Opportunities of Efficient and Performant Storage Systems (CHEOPS)*, 2026.

[6] T. Bingmann, "TLX: Collection of sophisticated C++ data structures, algorithms, and miscellaneous helpers," https://github.com/tlx/tlx, 2018.

[7] SQLite developers, "SQLite," https://www.sqlite.org.

[8] A. Galakatos, M. Markovitch, C. Binnig, R. Fonseca, and T. Kraska, "FITing-Tree: A data-aware index structure," in *Proceedings of the 2019 International Conference on Management of Data (SIGMOD)*, 2019.

[9] J. Ding, U. F. Minhas, J. Yu, C. Wang, J. Do, Y. Li, H. Zhang, B. Chandramouli, J. Gehrke, D. Kossmann, D. Lomet, and T. Kraska, "ALEX: An updatable adaptive learned index," in *Proceedings of the 2020 International Conference on Management of Data (SIGMOD)*, 2020.

[10] A. Itai, A. G. Konheim, and M. Rodeh, "A sparse table implementation of priority queues," in *Automata, Languages and Programming (ICALP)*, 1981.
[11] M. A. Bender, E. D. Demaine, and M. Farach-Colton, "Cache-oblivious B-trees," in *41st Annual Symposium on Foundations of Computer Science (FOCS)*, 2000.
[12] J. O'Rourke, "An on-line algorithm for fitting straight lines between data ranges," *Communications of the ACM*, vol. 24, no. 9, 1981.
[13] P. Ferragina and G. Vinciguerra, "The PGM-index: a fully-dynamic compressed learned index with provable worst-case bounds," *Proceedings of the VLDB Endowment*, vol. 13, no. 8, 2020.
[14] A. Kipf, R. Marcus, A. van Renen, M. Stoian, A. Kemper, T. Kraska, and T. Neumann, "RadixSpline: A single-pass learned index," in *Proceedings of the Third International Workshop on Exploiting Artificial Intelligence Techniques for Data Management (aiDM)*, 2020.
[15] C. Wongkham, B. Lu, C. Liu, Z. Zhong, E. Lo, and T. Wang, "Are updatable learned indexes ready?" *Proceedings of the VLDB Endowment*, vol. 15, no. 11, 2022.
[16] H. Lan, Z. Bao, J. S. Culpepper, and R. Borovica-Gajic, "Updatable learned indexes meet disk-resident DBMS – from evaluations to design choices," *Proceedings of the ACM on Management of Data*, vol. 1, no. 2, 2023.
[17] E. M. Kornaropoulos, S. Ren, and R. Tamassia, "The price of tailoring the index to your data: Poisoning attacks on learned index structures," in *Proceedings of the 2022 International Conference on Management of Data (SIGMOD)*, 2022.
[18] M. Bachfischer, R. Borovica-Gajic, and B. I. P. Rubinstein, "Testing the robustness of learned index structures," arXiv:2207.11575, 2022.
[19] R. Yang, E. M. Kornaropoulos, and Y. Cheng, "Algorithmic complexity attacks on dynamic learned indexes," *Proceedings of the VLDB Endowment*, vol. 17, 2024.
[20] D. R. Slutz, "Massive stochastic testing of SQL," in *Proceedings of the 24th International Conference on Very Large Data Bases (VLDB)*, 1998.
[21] M. Rigger and Z. Su, "Testing database engines via pivoted query synthesis," in *14th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2020.
[22] T. Mytkowicz, A. Diwan, M. Hauswirth, and P. F. Sweeney, "Producing wrong data without doing anything obviously wrong!" in *Proceedings of the 14th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2009.